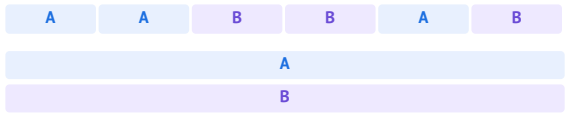
Конкурентность, потоки и asyncio

память и GIL · CPU / I/O · гонки и Lock · корутины и event loop
задачи · блокирующий код · лимиты запросов и ошибки

01 Конкурентность и параллелизм

Конкурентность: несколько задач в работе за один период. **Параллелизм:** задачи выполняются одновременно.



CPU-bound: долго считает. **I/O-bound:** долго ждёт сеть, БД или диск. Для I/O конкурентность перекрывает ожидания.

02 Процессы, потоки и GIL

	ПОТОКИ	ПРОЦЕССЫ
Память	общая	изолирована
Обмен	общие объекты	IPC / сериализация
Запуск	обычно дешевле	обычно дороже

GIL в обычном CPython: один поток интерпретатора выполняет Python-байткод за раз. При блокирующем I/O GIL освобождается; у процессов свои интерпретаторы.

С Python 3.13 есть отдельные сборки без GIL. Некоторые нативные библиотеки отпускают GIL и при вычислениях.

03 Что выбрать под задачу

НАГРУЗКА	ИНСТРУМЕНТ
I/O + async-библиотека	asyncio
I/O + sync-библиотека	ThreadPoolExecutor
CPU: чистый Python	ProcessPoolExecutor

100 сетевых запросов → asyncio + лимит + таймаут.
Частоты слов в корпусе → процессы + порции данных.

Запуск, передача и сбор данных тоже стоят времени. Маленький объём сначала делай последовательно.

04 Race condition → Lock

Два потока читают **5** и оба пишут **6**. Ожидали **7**: потерялось одно увеличение (**lost update**).

A: 5 → 6 B: 5 → 6 итог: 6 ≠ 7

```
lock = threading.Lock()  # один на всех
with lock:
    counter += 1
```

Lock на чтение, вычисление и запись; проверки тоже внутри. Всем потокам нужен **один lock**. GIL его не заменяет.

Deadlock: A держит L1 и ждёт L2, B - наоборот. Захватывай блокировки в одном порядке.

05 async / await и event loop

`async def f()`	описание корутины
`f()`	объект; ещё не запущен
`await f()`	выполнение и результат
`asyncio.run(main())`	вход в обычный скрипт



готовность I/O → снова в очередь готовых

В одном loop - **одна задача за раз**. При ожидании **await** отдаёт управление; готовый результат может вернуться **без переключения**.

06 Task, create_task и gather

Task организует выполнение корутины через loop.

```
a = asyncio.create_task(job("A"))
b = asyncio.create_task(job("B"))
x = await a
y = await b
```

Если job ждёт 1 с: **await A; await B ≈ 2 с**.
Обе Task созданы до ожидания: **≈ 1 с**.

gather(*aws) планирует корутины как задачи. Ответы идут **в порядке аргументов**. По умолчанию обычная ошибка передаётся вызывающему коду.

07 Блокирующий код в async

БЛОКИРУЕТ LOOP	ОТДАЁТ УПРАВЛЕНИЕ
`time.sleep(2)`	`await asyncio.sleep(2)`
остальные задачи ждут	остальные работают

async def не делает sync-код неблокирующим. Sync HTTP / БД, файловое I/O и долгий CPU-цикл занимают поток event loop.

```
result = await asyncio.to_thread(
    blocking_io, arg)
```

Для CPU на чистом Python - пул процессов.
to_thread нужен прежде всего для sync I/O.

08 Лимит запросов и ошибки

```
sem = asyncio.Semaphore(10)
async def one(url):
    async with sem:
        return await asyncio.wait_for(
            fetch(url), timeout=3)

results = await asyncio.gather(
    *(one(u) for u in urls),
    return_exceptions=True,
)
```

До **10 активных запросов**. Место освобождается и при ошибке. Таймаут здесь **не включает** ожидание семафора.

Лимит конкурентности ≠ запросов в секунду.

09 Результаты без общего счётчика

Каждый работник возвращает **локальный результат**, главный поток объединяет. Удобно для **независимых порций**.

```
with ThreadPoolExecutor() as pool:
    parts = pool.map(count, chunks)
total = sum(parts)
```

Ошибки gather: в results - ответы или исключения. Разбирай каждый результат; **CancelledError** обычно передавай выше.

Queue: потокобезопасная передача результатов одному сборщику. Общий изменяемый счётчик не нужен.

Оба прочитали 5 и записали 6?
Потерян +1. Lock на чтение, вычисление и запись.

Как собрать результаты?
Локальный return → sum.
Для потока ответов - Queue.

time.sleep внутри async?
Остановится поток loop; остальные задачи ждут.

Лимит + ошибка запроса?
Semaphore(N) + gather (return_exceptions=True).